SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Machine Learning

ASSESSMENT TOOL 1 – Experiments

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO3 – Precision	**Assessment:**	Assessment Tool 1 – Experiments
Weightage:	50% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	D. Sri Anjaneya	**Reg. No.:**	192425219
Section / Batch:	AI & ML / 2024	**Date:**	06/08/2026

Code of Conduct

I _D. Sri Anjaneya / 192425219_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: __D. Sri Anjaneya__

Instructions

- Perform all experiments using Google Colab.
- Create a separate notebook (.ipynb) for the assessment.
- Ensure all code is executable without errors.
- Include appropriate comments explaining the logic used.
- Complete all three experiments in a single Google Colab notebook.
- Execute all cells and display the outputs for the given test cases.
- Save the notebook with the naming convention: RegNo_Name_CO3-AT1.ipynb
- Upload the notebook to your GitHub repository.
- Ensure the repository is public or accessible to the instructor.
- Submit the following:
- GitHub Repository Link in GCR
- Google Colab Share Link in GCR
- Screenshots of uploaded coding and output files as printout.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
Context-Free Grammars for English Syntax, Context-Free Rules and Trees, Sentence-Level Constructions, Parsing, Top-Down Parsing, Earley Parsing	Design a Python program to validate sentences using CFG production rules and construct parse tree representations for valid sentences.	Q1

Agreement, Feature Structures, Sub Categorization	Design a Python program to verify subject–verb agreement and validate grammatical constraints using feature structures and subcategorization rules.	Q2
Probabilistic Context-Free Grammars (PCFGs)	Design a Python program to parse sentences and compute sentence probabilities using probabilistic grammar production rules.	Q3

Annexure A – Experiments

Experiment 1: Context-Free Grammar Rule Validation and Parse Tree Construction

Experiment Description

Design a Python program to validate whether a sentence follows the given Context-Free Grammar (CFG) rules and generate its parse tree representation.

Grammar:

```
S  → NP VP
NP → Det N
VP → V NP
Det → the | a
N   → student | teacher | book
V   → reads | likes
```

Task

1. Accept a sentence as input.
2. Verify whether it conforms to the CFG.
3. Construct and return the parse tree.
4. Return "Invalid Sentence" if parsing fails.

Function Prototype

```
def parse_cfg(sentence):
    pass
```

Test Cases

Input	Expected Output
the student reads a book	Valid Parse Tree
a teacher likes the book	Valid Parse Tree
student reads book	Invalid Sentence
the book likes a teacher	Valid Parse Tree
reads the student book	Invalid Sentence

Experiment 2: Agreement and Feature Structure Checking

Experiment Description

Design a Python program that verifies subject–verb agreement using feature structures containing Number and Person attributes.

Feature Rules

```
he      → singular, third person she    →
singular, third person it
→ singular, third person they
→ plural
```

```
runs   → singular
verb writes →
singular verb

run    → plural verb
write  → plural verb
```

Task

1. Accept Subject and Verb.
2. Compare feature structures.
3. Return True if agreement holds.
4. Return False otherwise.

Function Prototype

```
def check_agreement(subject, verb):
    pass
```

Test Cases

Subject	Verb	Expected
he	runs	True
he	run	False
they	run	True
they	writes	False
she	writes	True

Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing

Experiment Description

Implement a Python program to compute the probability of a sentence using the given PCFG.

Grammar

```
S  → NP VP      [1.0]
NP → John       [0.6]
NP → Mary       [0.4]
VP → runs       [0.5]
VP → walks      [0.5]
```

Probability Formula

```
P(Sentence)
=
P(S→NP VP)
× P(NP)
× P(VP)
```

Task

1. Accept a two-word sentence.
2. Parse using the PCFG.
3. Compute sentence probability.
4. Return 0 if sentence cannot be generated.

Function Prototype

```
def sentence_probability(sentence):
    pass
```

Test Cases

Input Sentence	Expected Probability
John runs	0.30
John walks	0.30
Mary runs	0.20
Mary walks	0.20
Peter runs	0.00

Experiment 1: Context-Free Grammar (CFG) Rule Validation and Parse Tree Construction

1. Explain the purpose of a Context-Free Grammar (CFG) in Natural Language Processing and Compiler Design.

A **Context-Free Grammar (CFG)** is a collection of production rules used to describe the grammatical structure of a language. It specifies how words or symbols can be combined to form valid sentences.

Purpose of CFG:

- Defines the syntax of a language.
- Verifies whether a sentence follows grammatical rules.
- Generates parse trees for valid sentences.
- Used in compiler design for syntax analysis.
- Used in Natural Language Processing (NLP) for sentence parsing and language understanding.

2. Role of Each Non-Terminal Symbol

Non-Terminal	Role
S	Represents the complete sentence (Start Symbol).
NP	Represents a Noun Phrase consisting of a Determiner and a Noun.
VP	Represents a Verb Phrase consisting of a Verb followed by a Noun Phrase.
Det	Represents a Determiner such as **the** or **a**.
N	Represents a Noun such as **student**, **teacher**, or **book**.
V	Represents a Verb such as **reads** or **likes**.

3. Validate the Given Sentences

(a) the student reads a book

Structure:

the student reads a book

NP V NP

Grammar:

S → NP VP

NP → Det N

VP → V NP

Result: ☑ Valid Sentence

(b) a teacher likes the book

Structure:

a teacher likes the book

NP V NP

Result: ☑ Valid Sentence

(c) student reads book

The nouns **student** and **book** do not have determiners (**the** or **a**).

Required grammar:

NP → Det N

Result: ❌ Invalid Sentence

(d) the book likes a teacher

Structure:

the book likes a teacher

NP V NP

The sentence follows the grammar rules even though its meaning is unusual.

Result: ☑ Valid Sentence

(e) reads the student book

The sentence begins with a verb, but according to the grammar a sentence must begin with a noun phrase.

Required grammar:

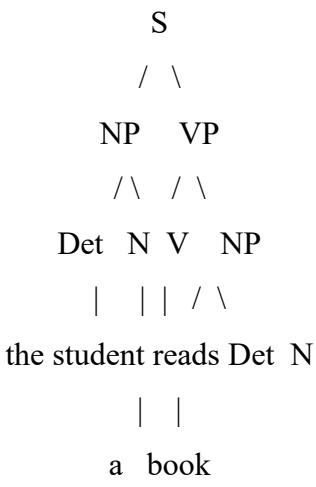
S → NP VP

Result: ❌ Invalid Sentence

Validation Summary

Sentence	**Result**
the student reads a book	Valid Sentence
a teacher likes the book	Valid Sentence
student reads book	Invalid Sentence
the book likes a teacher	Valid Sentence
reads the student book	Invalid Sentence

4. Parse Tree for "the student reads a book"



Conclusion

The given grammar correctly validates sentences that follow the production rules. Valid sentences produce a parse tree representing their grammatical structure, while sentences that violate the grammar are identified as **Invalid Sentence**.

Experiment 2: Agreement and Feature Structure Checking

1. What is Subject–Verb Agreement?

Subject–verb agreement is the grammatical rule that the verb must agree with the subject in **number** (singular or plural) and **person** (first, second, or third).

For example:

- **He runs.** ✔
- **They run.** ✔
- **He run.** ✘
- **They writes.** ✘

2. Feature Structures

Subject Features

Subject	Number	Person
he	Singular	Third Person
she	Singular	Third Person

Subject	Number	Person
it	Singular	Third Person
they	Plural	Third Person

Verb Features

Verb	Number
runs	Singular
writes	Singular
run	Plural
write	Plural

3. Check Subject–Verb Agreement

(a) Subject: he Verb: runs

- Subject → Singular, Third Person
- Verb → Singular

Both features match.

Result: ☑ True

(b) Subject: he Verb: run

- Subject → Singular
- Verb → Plural

The features do not match.

Result: ❌ False

(c) Subject: they Verb: run

- Subject → Plural
- Verb → Plural

Both features match.

Result: ☑ True

(d) Subject: they Verb: writes

- Subject → Plural
- Verb → Singular

The features do not match.

Result: ❌ False

(e) Subject: she Verb: writes

- Subject → Singular, Third Person
- Verb → Singular

Both features match.

Result: ☑ True

4. Test Case Summary

Subject	Verb	Result
he	runs	True
he	run	False
they	run	True
they	writes	False
she	writes	True

Conclusion

Subject–verb agreement ensures that the subject and the verb have matching grammatical features. A sentence is considered **correct (True)** when the subject and verb agree in number and person; otherwise, it is **incorrect (False)**.

Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing

1. What is a Probabilistic Context-Free Grammar (PCFG)?

A **Probabilistic Context-Free Grammar (PCFG)** is an extension of a Context-Free Grammar (CFG) in which each production rule is assigned a probability. These probabilities help determine how likely a sentence is to be generated by the grammar.

A PCFG is commonly used in **Natural Language Processing (NLP)** for parsing ambiguous sentences and selecting the most probable parse tree.

2. Given PCFG

S → NP VP [1.0]

NP → John [0.6]

NP → Mary [0.4]

VP → runs [0.5]

VP → walks [0.5]

3. Probability Formula

The probability of a sentence is calculated as:

P(Sentence) = P(S → NP VP) × P(NP) × P(VP)

4. Calculate Sentence Probability

(a) John runs

P = 1.0 × 0.6 × 0.5

= 0.30

Result: ☑ 0.30

(b) John walks

P = 1.0 × 0.6 × 0.5

= 0.30

Result: ☑ 0.30

(c) Mary runs

P = 1.0 × 0.4 × 0.5

= 0.20

Result: ☑ 0.20

(d) Mary walks

P = 1.0 × 0.4 × 0.5

= 0.20

Result: ☑ 0.20

(e) Peter runs

The grammar does not contain the rule:

NP → Peter

Therefore, the sentence cannot be generated by the grammar.

Result: ✗ 0.00

5. Test Case Summary

Input Sentence	Probability
John runs	0.30
John walks	0.30
Mary runs	0.20
Mary walks	0.20
Peter runs	0.00

Conclusion:

A **Probabilistic Context-Free Grammar (PCFG)** assigns probabilities to grammar rules, allowing the likelihood of a sentence to be calculated. The probability of a valid sentence is obtained by multiplying the probabilities of the production rules used to generate it. If a sentence cannot be generated using the given grammar, its probability is **0.00**.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____